



S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Practical 07

Aim: Develop a program to manage resource allocation for five processes (Google Drive, Firefox, Word Processor, Excel, and PowerPoint) using four types of resources (Printer, ROM, Hard Disk, and RAM). The program takes input for allocated, maximum, and available resources, calculates the current need of each process, and determines if a safe execution order exists using the Banker's Algorithm.

Name: Ayushi Manoj Kukde

USN: EE24TCD001

Semester / Year: 4th / 2nd

Academic Year: 2025-2026

Date of Performance:

Date of Submission:

❖ CODE :

M ~

```
GNU nano 8.7
#include <stdio.h>

int main() {
    int i, j, k;
    int alloc[5][4], max[5][4], need[5][4];
    int avail[4], finish[5] = {0}, safe[5];
    int count = 0;

    // Input Allocation Matrix
    printf("Enter Allocation Matrix:\n");
    for(i = 0; i < 5; i++)
        for(j = 0; j < 4; j++)
            scanf("%d", &alloc[i][j]);

    // Input Maximum Matrix
    printf("Enter Maximum Matrix:\n");
    for(i = 0; i < 5; i++)
        for(j = 0; j < 4; j++)
            scanf("%d", &max[i][j]);

    // Input Available Resources
    printf("Enter Available Resources:\n");
    for(j = 0; j < 4; j++)
        scanf("%d", &avail[j]);

    // Calculate Need Matrix
    for(i = 0; i < 5; i++)
        for(j = 0; j < 4; j++)
            need[i][j] = max[i][j] - alloc[i][j];

    // Banker's Algorithm
    while(count < 5) {
        int found = 0;

        for(i = 0; i < 5; i++) {
            if(finish[i] == 0) {
                for(j = 0; j < 4; j++)
                    if(need[i][j] > avail[j])
                        break;

                if(j == 4) {
                    for(k = 0; k < 4; k++)
                        avail[k] += alloc[i][k];

                    safe[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }

        if(found == 0) {
            printf("\nSystem is NOT in safe state\n");
            return 0;
        }
    }

    // Output Safe Sequence
    printf("\nSystem is in SAFE state\nSafe sequence: ");
    for(i = 0; i < 5; i++)
        printf("%d ", safe[i]);

    return 0;
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^N Replace

^K Cut
^U Paste

❖ OUTPUT :

 ~

```
De1l@DESKTOP-D7MT61A UCRT64 ~  
$ nano bankers.c  
  
De1l@DESKTOP-D7MT61A UCRT64 ~  
$ nano banker.c  
  
De1l@DESKTOP-D7MT61A UCRT64 ~  
$ gcc banker.c -o banker  
  
De1l@DESKTOP-D7MT61A UCRT64 ~  
$ ./banker  
Enter Allocation Matrix:  
0 1 0 3  
2 0 0 1  
3 0 2 1  
2 1 1 0  
0 0 2 2  
Enter Maximum Matrix:  
7 5 3 4  
3 2 2 2  
9 0 2 2  
2 2 2 2  
4 3 3 3  
Enter Available Resources:  
3 3 2 2  
  
System is in SAFE state  
Safe sequence: P1 P3 P4 P0 P2  
De1l@DESKTOP-D7MT61A UCRT64 ~  
$ |
```